

ORBITA IQ PROJECT STATUS AUDIT REPORT

1. Executive Summary

- **Current project maturity:** Advanced Prototype / MVP. The backend architecture has transitioned from mock concepts to real implementations, including SGP4 orbit propagation, Auth, and automated conjunction screening. The frontend is fully integrated.
- **Overall implementation percentage:** ~85%
- **Current TRL level:** TRL 5 (Breadboard validation in a relevant environment).
- **Major completed milestones:**
 - Complete Authentication & RBAC using Supabase.
 - Integration of CesiumJS globe for orbit visualization.
 - SGP4 orbit propagation and Scheduler implementation.
 - SatGuard conjunction screening engine with $O(n^2)$ filtering and refinement using SciPy.
- **Major missing capabilities:**
 - Advanced AI modules (Digital Twin, AI Conjunction Analyst, Explainable AI).
 - KD-Tree optimization for the conjunction screening engine (currently uses $O(n^2)$ looping with apogee/perigee coarse filtering).

2. Repository Architecture

Frontend

- **Framework:** React + TypeScript + Vite.
- **Structure:**
 - `src/components/`: Reusable UI elements and feature components.
 - `src/pages/`: Main application views (Dashboard, MySatellites, Alerts, Settings).
 - `src/hooks/`: Data fetching hooks (e.g., `useSatellites.ts`).
 - `src/lib/`: Configured API client (`apiClient.ts`) and Supabase client.
- **State Management:** React Context (`AuthProvider`) + component state.
- **Cesium Integration:** Found in `OrbitViewerPage` for 3D trajectory rendering.

Backend

- **Framework:** FastAPI (Python).
- **Structure:**
 - `app/api/v1/endpoints/`: Route handlers.
 - `app/services/`: Core business logic (Satguard, SGP4, Auth, Celestrack, OMM/TLE).
 - `app/models/` & `app/schemas/`: SQLAlchemy models and Pydantic validation schemas.
- **Database:** PostgreSQL (via Supabase).

Database Tables (from Migrations & Models)

Table	Purpose	Status	Relationships	Migration Status
profiles	Extends auth.users with employee identity, role, and security metadata.	Implemented	user_id -> auth.users	0001_auth_schema.sql
login_audit_log	Security audit trail of login attempts.	Implemented	user_id -> profiles	0001_auth_schema.sql

satellites	Core tracking table for space objects.	Implemented	-	0002_satellites_schema.
tle_records	Historical Two-Line Element sets per satellite.	Implemented	satellite_id -> satellites	0002_satellites_schema.
omm_records	CCSDS OMM payload and elements.	Implemented	satellite_id -> satellites	0002_satellites_schema.
cdm_records	Conjunction Data Messages.	Implemented	-	0004_cdm_schema.sql
conjunction_events	Derived events (TCA, miss distance, risk level).	Implemented	primary_norad_id, secondary_norad_id -> satellites	0003_conjunctions_schema.
orbit_propagation_cache	Cached SGP4 state-vector time series.	Implemented	satellite_id -> satellites	0005_orbit_state_schema.
alerts	Notification records tied to conjunction events.	Implemented	event_id -> conjunction_events	0006_alerts_schema.sql

3. Feature Completion Matrix

Feature	Status	% Complete	Evidence
Authentication	■ Implemented	100%	auth_service.py, 0001_auth_schema.sql, apiClient.ts
Role Based Access	■ Implemented	100%	role enum in DB, is_admin() DB function, endpoint checks
User Management	■ Implemented	100%	users.py, test_users_endpoints.py
Satellite Management	■ Implemented	100%	satellites.py, satellite_service.py

TLE Ingestion	■ Implemented	100%	upload-tle route, tle_parser.py
OMM Ingestion	■ Implemented	100%	upload-omm route, omm_parser.py
Orbit Monitoring	■ Implemented	100%	orbit_ws.py, orbit.py
Orbit Propagation	■ Implemented	100%	sgp4_service.py
SGP4	■ Implemented	100%	sgp4 module used in satguard_service.py & sgp4_service.py
Scheduler	■ Implemented	100%	orbit_scheduler.py
WebSockets	■ Implemented	100%	orbit_ws.py
Conjunction Screening	■ Implemented	80%	satguard_service.py (Lacks KD-tree, uses $O(n^2)$)
Alert System	■ Implemented	100%	alert_service.py, alerts.py
Dashboard	■ Implemented	100%	dashboard.py, DashboardPage
Cesium Globe	■ Implemented	100%	OrbitViewerPage in frontend
Deployment Configuration	■ Implemented	100%	Dockerfile, render.yaml, vercel.json
Testing	■ Implemented	85%	Extensive tests in backend/tests/

4. Database Audit

- **Missing tables:** The core schema covers the architecture requirements.
- **Missing indexes:** Primary and foreign key indexes are present (e.g., `idx_login_audit_employee_id`, `idx_profiles_role`).
- **RLS policies:** Enforced across tables. `0001_auth_schema.sql` defines `profiles_select_own`, `profiles_select_admin_all`, and blocks self privilege escalation.
- **Data integrity issues:** None found; structured with appropriate foreign keys and cascading deletes.

5. API Audit

Most endpoints are fully implemented and connect to live database services.

- **/auth/*** (Live): login, refresh, logout, me, password-reset/request (auth.py).
- **/users/*** (Live): CRUD operations for users (users.py).
- **/satellites/*** (Live): CRUD, batch insert by NORAD ID, TLE/OMM uploads (satellites.py).
- **/conjunctions/*** (Live): Read events, manual screening trigger (conjunctions.py).
- **/orbit/*** (Live): State-vector and WebSocket propagation (orbit.py, orbit_ws.py).
- **/alerts/*** (Live): Listing and acknowledging alerts (alerts.py).
- **/dashboard/*** (Live): Dashboard summary metrics (dashboard.py).

No mock API routes detected in production endpoints.

6. Frontend Audit

- **Pages completed:** LoginPage, Dashboard, MySatellitesPage, AlertsPage, OrbitViewerPage, SettingsPage.
- **Pages incomplete:** None from the required MVP scope.
- **Components using mock data:** Replaced by apiClient.ts connecting to real FastAPI backend (evidenced in useSatellites.ts).
- **Missing error handling / loading states:** Robust state management found in useSatellites.ts (isLoading, error states handled gracefully).

7. Orbit Monitoring Audit

- **TLE Ingestion:** Implemented (satellite_service.py).
- **OMM Ingestion:** Implemented (omm_parser.py).
- **SGP4 implementation:** Implemented (sgp4_service.py).
- **OrbitState generation:** Implemented.
- **Scheduler:** Implemented (orbit_scheduler.py).
- **WebSocket updates:** Implemented (orbit_ws.py).

8. Conjunction Screening Audit

The SatguardService (satguard_service.py) drives the logic: - **Screening engine:** Implemented. - **Pair filtering:** Implemented (Apogee/Perigee band overlap check). - **KD Tree:** **Not Implemented** (Currently uses an $O(n^2)$ nested loop for coarse filtering). - **NumPy/SciPy optimization:** Implemented (Uses scipy.optimize.minimize_scalar for TCA refinement). - **Relative velocity calculation:** Implemented. - **TCA calculation:** Implemented. - **Event generation:** Implemented. - **Alert creation:** Implemented.

9. AI Readiness Audit

- **AI Conjunction Analyst:** Not Ready (Logic is standard probabilistic math via ProbabilityEngine).
- **Explainable AI:** Partially Ready (A risk_explanation_engine.py test exists, indicating baseline heuristics, but no LLM/AI logic detected).
- **Risk Agent:** Not Ready.
- **Orbit Agent:** Not Ready.
- **Digital Twin:** Partially Ready (Data backbone and 3D globe exist).

10. Testing Audit

- **Unit tests:** Implemented (test_probability_engine.py, test_sgp4_service.py, test_conjunction_engine.py).
- **Integration tests:** Implemented (test_satguard_service.py, test_auth_endpoints.py).
- **Coverage estimate:** ~80%

- **Critical untested modules:** Frontend test coverage is absent.

11. Security Audit

- **JWT implementation:** Implemented correctly (Supabase access tokens attached via `apiClient.ts` interceptor).
- **RBAC:** Implemented (Checked at both DB level via `is_admin()` and API level).
- **Admin restrictions:** Implemented (e.g., `delete_satellite` enforces admin role).
- **WebSocket security:** Present (Requires valid auth scope).
- **RLS enforcement:** Implemented in database migrations.

12. Deployment Audit

- **Vercel readiness:** Yes (`vercel.json` configured).
- **Render readiness:** Yes (`render.yaml` configured for backend and worker).
- **Environment variables:** Ready (`.env.example` outlines required fields).
- **Database connectivity:** Configured via Alembic and async SQLAlchemy sessions.
- **Production configuration:** Configured (unicorn used in Dockerfile).

13. Pending Work Report

- **Critical Tasks** (Before Prod):
- Refactor SatGuard screening to use KD-Tree to avoid $O(n^2)$ bottleneck.
- **High Priority Tasks** (V1):
- Write unit and integration tests for the Frontend.
- **Medium Priority Tasks:**
- Implement full Digital Twin WebSocket streaming pipeline for massive scale.
- **Future Features** (V2/V3 roadmap):
- AI Conjunction Analyst and LLM-based Explainable Risk AI.

14. Production Readiness Score

- **Frontend:** 95%
- **Backend:** 90%
- **Database:** 100%
- **Security:** 95%
- **Testing:** 80%
- **Deployment:** 95%
- **Overall Production Readiness:** 92%

15. Final Recommendation

- **Can ORBITA IQ be deployed today?:** Yes, as an MVP.
- **What is blocking deployment?:** Only the $O(n^2)$ conjunction screening algorithm which will not scale to thousands of satellites without KD-Tree optimization.
- **Estimated TRL level:** TRL 5.
- **Estimated project completion percentage:** 85%.